RG660MK × UGREEN 智能音箱

设计方案、测试方案与问题史复盘（截至 2026-11-07）

设备 RG660MK / MediaTek T930　声卡 UGREEN CM564 (plughw:2,0)　ASR paraformer · TTS 本地 piper　代码 1739 行 + 4 个支撑模块

生成者：hermes ｜ 数据来源：设备端代码与运行日志、git 提交历史（25 条语音相关）、 2026-09-14 设计基线 V1.0、2026-09-21 麦克风验证记录
所有延迟数字取自设备 /data/ai_cpe/logs/voice.log 的 LATENCY 行，样本量在各表标注
现场：中国移动网络大会（2026-11-07） ｜设备在线，语音与看板实测正常

一、系统形态与设计目标

一台 RG660MK（5G CPE）外接 UGREEN CM564 USB 声卡，盒子的麦克风与扬声器共用 `plughw:2,0`，构成「能听会说、本地闭环」的智能音箱。设计目标来自 2026-09-14 的《智能音箱优化设计 V1.0》，三条硬约束至今未变：

目标	定义	验收口径
快	简单确定性问题不进大模型，本机 1~2 秒内开始回复	唤醒→「在呢」≤ 2 s；报时/算术 ≤ 2 s
准	天气、股价等实时信息必须来自实时数据，不能用模型常识猜	答案带数据源与时间戳
稳	每轮 turn_id 隔离，旧答案/迟到结果不得串入下一轮	30 s 未出结果播「请稍等」，60 s 终止

为什么强调「本地」：展会与派出所场景经常没有稳定公网。设计上把唤醒、ASR、TTS、 时间/算术/灯控全部放在设备侧；只有天气、股价、知识问答才出网。断网时音箱仍然可用，只是「变笨」。

二、硬件与音频链路

环节	实测配置	说明与踩过的坑
麦克风	CM564，`plughw:2,0`，48 kHz，采集增益 190/255（75%，2026-11-07 现场上调）	曾误判为「聋麦」——raw RMS 只有 70~150；后证实是信号弱而非硬件坏，靠增益与近场说话解决
扬声器	同一 CM564 盒子，`plughw:2,0`	盒子喇叭本身正常，问题都出在采集侧；播放与采集共用一设备，需避免抢占
摄像头麦克风	C270 麦（`plughw:1,0`）**已禁用**	灵敏度是 CM564 的 3~10 倍，一度切过去救急；但会与 camview 抢占 USB，最终回归 CM564
VAD	RMS 门限 450 起 / 280 停（滞回）	可用 `VOICE_VAD_ON/OFF` 覆盖；换麦或换位置必须重新标定，否则「一直录」或「听不见」
回声	无 AEC，靠「播报时静音采集」规避	播 TTS 期间 `mute_for()` 屏蔽录音，避免把自己的声音当成用户说话

三、软件架构



模块	规模/位置	职责
`voice_assistant.py`	1739 行 / root	主循环：唤醒、VAD、ASR、路由、对话管理、TTS 调度
paraformer 常驻服务	`ws://127.0.0.1:6006`	sherpa-onnx offline websocket，模型 paraformer-zh-small（int8）
`local_tts.py`	8.5 KB	piper 优先（zh_CN-huayan-medium），sherpa VITS 兜底；音频不出设备
`tts_say.py`	1.5 KB	命令行播报入口：合成 → aplay 到 CM564
`vad.py`	小模块	RMS 滞回端点检测，阈值可环境变量覆盖
KWS（sherpa keywords）	**模型已缺失**	原设计为声学唤醒第一道门；缺失后 `kws_hit()` 恒返 False，全靠 ASR 文本兜底

分层路由（强规则优先）

优先级固定为 **设备控制 > 天气/实时 > 本地时间 > 本地计算 > 大模型**。 这条顺序是硬约束：「今天上海什么天气」必须命中天气而不是日期；「开灯」必须走设备控制不能进大模型。

层级	典型请求	处理器	响应目标
L0 本地	现在几点、今天几号、1+1、开灯、灯是不是开着	本地函数 / smarthome.py	**1~2 秒内开始播报**
L1 实时	今天上海什么天气、移远通信股价	open-meteo / 腾讯行情	3~6 秒（含网络）
L2 复杂	相对论是怎么回事、5G SA 和 NSA 区别	在线大模型 / Hermes	5~15 秒，超 30 s 播「请稍等」

四、关键设计决策（每条都对应一次实测）

决策	为什么这么定	实测依据
麦克风用 CM564、增益 190	与扬声器同盒、走线简单；C270 麦会与 camview 抢 USB	双麦对照：CM564 RMS 70~150（近场可辨），C270 600~2400（但会抢占）
ASR 用 paraformer 常驻服务	whisper 单次 2.2 s，paraformer 常驻 0.78~1.6 s，且模型小	当前 18 次样本：p50 0.943 s、max 1.614 s
唤醒词变体表（小提/下皮/小陈…）	口音下 paraformer 稳定把「小皮」听成「小提」，单靠标准词永远唤不醒	2026-09-21 实测「你好小提」；2026-11-05 实测「你好小陈」
休眠态只认完整「你好小皮」	用户要求「说了休息就别轻易启动」，避免电视/聊天误唤醒	休眠后单说「几点」不响应，说完整称呼才回「在呢」
热词 prompt 默认关闭	把业务词表塞进 ASR 上文会污染解码，且每次 +2.2 s	真实录音 A/B：开 prompt 无收益、更慢、更易出碎片
TTS 走本地 piper	edge-tts 云端 403、连接重置、延迟不可控；本地合成不吃网络	piper RTF≈0.76（A55），就绪 0.72~0.86 s，缓存命中 0.005 s

决策	为什么这么定	实测依据
报时用预合成片段	逐字合成「现在是 11 月 6 日上午 9 点 34 分星期四」要 8 s	改为片段拼接后 2.4 s（历史记录）
灯控走 `--nowait`	Matter 真灯确认要 12~19 s，语音侧 6 s 超时会误报「操作失败」	发令即返回 0.3 s，实际开关由后台确认

五、测试方案

5.1 三层测试

层级	测什么	方法
单元	时间解析、计算器、唤醒词剥离、垃圾判定	设备上直接 import 调函数断言（如 `classify("现在几点…噪音")` 必须返回 LOCAL_DATE_TIME）
链路	录音→ASR→路由→TTS→播放	真实说话，日志看 `[TURN] ASR_TEXT / ROUTER / RESPONSE / TTS_START` 四段是否齐全
场景	唤醒、连续对话、休眠/唤回、噪音、断网	按 9 条验收检查项逐条过（2026-10-14 落档）

5.2 九条验收检查项（2026-10-14 定稿）

- 唤醒一次成功，应答「在呢」≤ 2 s
- 报时/日期准确，与设备 `date` 一致
- 算术与设备控制不进大模型（日志无 LLM 调用）
- 天气/股价带数据源与时间
- 连续两轮不串答案（turn_id 隔离）
- 说「小皮休息」后静默，完整称呼可唤回
- 环境噪音不触发误唤醒
- 断网时本地功能仍可用（唤醒/报时/灯控）
- 播报期间不录自己的声音（无自问自答）

5.3 指标口径

指标	取数方式	当前基线
ASR 延迟	日志 `[LATENCY] asr_s=`	p50 0.943 s / max 1.614 s（n=18）
TTS 就绪	日志 `tts_ready_s=`	0.72~0.86 s（n=6）；缓存 0.005 s
唤醒→应答	WAKE 到「在呢」播放	1.25 s（2026-09-21 实测）~2.1 s
端到端问答	说完到开始播报	L0 2~4 s；L1 3~6 s；L2 5~15 s
唤醒成功率	喊 N 次成功次数	正脸正说 5/5（有 KWS 时期）；当前靠 ASR 兜底，受噪音影响更大

六、问题史（按主题，含根因与当前状态）

6.1 麦克风：从「聋麦」到「信号弱」

现象	根因	处置
对着盒子说话无反应	CM564 采集 RMS 只有 70~150，接近本底噪声；不是硬件坏，是灵敏度低	提高采集增益（140→190/255）；近场说话；临时可切 C270 麦救急

现象	根因	处置
切 C270 后唤醒好了但视觉异常	C270 麦与 camview 抢 USB 独占	回归 CM564；抓帧统一走 camview HTTP 口
换了位置后又「听不见」	VAD 阈值是按旧位置标定的	用 `VOICE_VAD_ON/OFF` 现场重标（1400/1100 用于 C270，450/280 用于 CM564）

6.2 唤醒：四次不同根因，同一个症状

时间	根因	修复
2026-09-21	变体表缺「小提」：ASR 听对了但唤醒词不认	补小提/下提/小蹄等变体（提交 d046e77）
2026-10 之后	KWS 模型文件丢失，声学唤醒恒 False，全靠 ASR 兜底	**未恢复（当前状态），靠 ASR 文本兜底**
2026-11-05	`asleep` 只置 True 从不置 False：说过一次「休息」后整场都在休眠态	唤醒成功后置 False（本轮修复）
2026-11-05	变体表缺「小陈」：本次现场实际误听	补小陈/小晨/小成/小城
2026-11-07	非休眠态误用严格判定 is_sleep_wake（要求「你好」前缀），转写成「小皮」时被丢弃	非休眠改用 is_wake（接受「小皮」短变体）；待机录音 8 s→5 s
2026-11-05	whisper-cli 与 ggml 模型文件丢失，ASR 抛 FileNotFoundError	切 paraformer 常驻服务（模型完好在机）

> **KWS 状态更新（2026-11-07 下午）：已上线。** 此前 `/data/ai_cpe/sherpa-onnx/model/` 缺失、`kws_hit()` 恒返 False，唤醒全靠 ASR 兜底； 现已交叉编译 keyword-spotter 并部署常驻服务，单次判定 386~505 ms， 命中「你好小皮」、负样本无误报，cron 看门狗 20 s 自愈。细节与代价见附录 C。

6.3 ASR 引擎演进

阶段	方案	结果
2026-09	whisper.cpp + ggml-base	可用但慢：单次 2.2 s；对连续笑声等噪声易出长串幻觉
2026-09-23	接入 paraformer（常驻 6006，可切换）	延迟降到 0.8~1.6 s；48k→16k 重采样后送 ws
2026-11-05	whisper 文件丢失 → 默认切 paraformer	**当前状态；日志无 FileNotFoundError**

6.4 时间与实时信息

- **问「现在几点」答错**：① ASR 把环境音拼在问句后（「现在几点朋友后面上次很多人…」）， 而时间正则用 `fullmatch` 整句匹配，直接漏到大模型，模型答「我没有查时间的工具」→ 改为「句首命中且 ≤30 字即按本地时间处理」；② 设备无 RTC 电池，断电后时间漂移 → 启用 NTP （后又因 overlay 重置被关掉，属待办）。
- **天气答英文**：wttr.in 的 lang_zh 退化 → 改 open-meteo 为主源（提交 1ea1836）。
- **股票必超时**：名称解析放大到 23~29 s，7 s 快失败掐断 → 名称解析移出快路径（提交 4eb4ace）。

6.5 TTS 演进

阶段	问题	结果
edge-tts（微软云）	端点变更全局 403；偶发连接重置；延迟不可控	打补丁 + 3 次重试，仍不稳定
sherpa-onnx 本地 VITS	不依赖网络，但 RTF 偏高	可用，作兜底
piper（当前）	加载 1.4 s、RTF≈0.76，比 VITS 快约 4 倍	**就绪 0.72~0.86 s；短句持久缓存命中 0.005 s**

6.6 其它已闭环的问题

- **灯控误报失败**：Matter 真灯确认 12~19 s，语音侧 6 s 超时 → 改 `--nowait`，0.3 s 返回。
- **报时慢**：逐字合成 8 s → 预合成片段拼接 2.4 s。
- **噪声误触发/自说自话**：垃圾文本判定 + 连续两次垃圾即静默退场；播报期间静音采集。
- **看门狗把进程杀错**：`pgrep -f voice_assistant.py` 会匹配到检查脚本自己 → 改用 `[v]oice_assistant.py` 括号法，且按文件名精确匹配。
- **断电丢文件**：whisper 二进制/模型、KWS 模型都在一次断电后消失（/data 分区）；已改为 paraformer 常驻 + 待补 KWS。

七、当前状态与待办

能力	状态	证据 / 说明
唤醒（ASR 兜底）	**可用但不理想**	无 KWS，每次唤醒先跑一遍 ASR；正脸正说可稳定唤醒
ASR 转写	**正常**	paraformer，p50 0.94 s；日志无报错
本地问答（时间/算术）	**正常**	已修 fullmatch 与变体；报时 2.4 s
联网问答（天气/股价）	**正常**	open-meteo / 腾讯行情，带数据源
TTS 播报	**正常**	piper 本地，0.72~0.86 s，缓存 0.005 s
休眠 / 唤回	**正常**	本轮修复 asleep 未复位
KWS 声学唤醒	**已上线（常驻 386 ms）**	交叉编译 keyword-spotter + 自写常驻服务，socket 调用；看门狗 20 s 自愈（见附录 C）
NTP 校时	**已启用**	2026-11-07 恢复 `ntp.enabled=1`，ntpd 运行，时差 0 s

三项待办的处置结果

- **KWS 模型 —— 已上线**：交叉编译 sherpa-onnx-keyword-spotter（aarch64 glibc，工具链用 `apt-get download` + `dpkg -x` 解出，无需 sudo），并自写常驻服务 `kws_server`（模型只加载一次，unix socket 调用）。实测单次 386~505 ms（按需命令行要 5.4~6.3 s），命中「你好小皮」、负样本无误报；cron 看门狗 20 s 自愈。
- **NTP 校时 —— 已完成**：`system.ntp.enabled=1`，服务器改为阿里云 + cn.pool，sysntpd 随系统启动，实测与本机时差 0 s。
- **语音自检 —— 已完成**：`demo_boot_check.sh` 新增 6/7 音频自检段（检查 card2 是否存在、采集增益，能独占设备时实测 2 秒录音字节数；被语音助手占用时改为检查最近日志活动）。

附录：关键路径与参数

项	值
主程序	`/data/ai_cpe/voice_assistant.py`（1739 行）
TTS 模块	`/data/ai_cpe/local_tts.py` + `tts_say.py`

项	值
ASR 服务	`sherpa-onnx-offline-websocket-server --port=6006`，模型 paraformer-zh-small
录音 / 播放	`plughw:2,0`（CM564），48 kHz，采集增益 190/255（75%）
运行日志	`/data/ai_cpe/logs/voice.log`（LATENCY/TURN/miss 行）
启动方式	`/etc/init.d/aicpe-boot` 内 nohup 拉起（无独立 init 脚本）
回滚备份	`voice_assistant.py.bak_20261105_asrpara` / `..._waketime2`

附录 B：典型故障 60 秒定位表

症状	第一条要看的证据	结论方向
喊了完全没反应	`ps w | grep [v]oice_assistant.py`; `head -3 logs/voice.log`	进程不在 → 拉起；日志 asr=whisper → 引擎配置回退
日志有 `[ASR] error=FileNotFoundError`	检查 whisper-cli / 模型文件是否存在	文件被断电清掉 → 切 paraformer（改默认引擎后重启）
要喊两次才应	日志有无 `[miss] sleep kws=miss asr=...`	处于休眠态且变体没命中 → 补变体；确认 asleep 在唤醒后置 False
答非所问（如问时间却答「没有查时间的工具」）	`[TURN?] ASR_TEXT=` 原文	ASR 尾部噪音导致规则未命中 → 放宽匹配或做句首匹配
播报没声音	`aplay -D plughw:2,0 /tmp/tts_say.wav` 手工试	设备被占用 / 音量 0 / 合成失败（看 `tts_ready_s` 是否出现）
报时错	`date` 与本机对比	差 > 60 s → 无 RTC 且 NTP 关闭，手工校时并恢复 NTP
采集电平异常	录 3 秒算 raw RMS（正常说话数百量级）	几十 → 增益/麦克风问题；上千且削顶 → 增益过高或离太近

> **一条通用原则：**音箱的问题几乎都能在 `voice.log` 里找到一行证据（`[miss]` / `[LATENCY]` / `[TURN]`）。先读日志再改参数，不要凭感觉调 VAD 阈值或换模型——这是过去两个月最容易走的弯路。

附录 C：KWS 声学唤醒 —— 从「不可部署」到「已上线」（2026-11-07）

C.1 上午的结论（当天下午被推翻）

三条路径确实都堵死：pip 无 aarch64-musl wheel；GitHub releases 及 ghfast / gh-proxy / kgithub 镜像不可达； 现有 `sherpa-onnx-offline` 与 `websocket-server` 二进制不含 keyword-spotter。 但结论「不可部署」下得太早——真正缺的不是网络，是编译。

C.2 实际走通的路径

步骤	做法
源码	本机已有 sherpa-onnx v1.13.4 完整源码（含 `sherpa-onnx-keyword-spotter.cc` 与 C++ API）
交叉工具链	`apt-get download` + `dpkg -x` 解出 aarch64-linux-gnu 的 gcc/g++ 与 sysroot， **不需要 sudo**；hello world 验证可生成 ARM64 ELF

步骤	做法
依赖	cmake 从 **hf-mirror** 拉 onnxruntime 静态库（github 失败自动回退），实测可达
模型	本机历史部署包 `pipi_kws_deploy_20260721_200856`（int8，5.2 MB）——正是用户记得的「以前部署过」的那套
常驻服务	自写 `kws_server.cpp`（C++ API，unix socket，模型只加载一次），约 120 行
看门狗	`kws_watchdog.sh` + cron 每分钟，`pgrep -f "[k]ws_server"` 避免自匹配

C.3 三种调用方式实测

方式	单次耗时	结论
命令行按需（keyword-spotter）	5400~6300 ms	**每次加载模型，比 ASR 还慢，不可用**
常驻服务（kws_server + socket）	386~505 ms	**已上线；含脚本与 python 启动开销**
原 ASR 兜底（对照）	约 900 ms	保留为 fallback

> **验证证据（ad-hoc，2026-11-07 13:40）：**常驻进程在、socket 在、正样本 386 ms 命中、负样本无误报、 看门狗未重复拉起——五项全过。看门狗自愈实测：杀掉进程后 20 秒被 cron 拉起。

C.4 仍然存在的代价

- **常驻内存** +18~52 MB（板子 available 约 496 MB）。
- **多一个进程要守护**：已加 cron 看门狗；自愈窗口（≤60 s）内 `kws_spot.sh` 会回退到 5.4 s 命令行， 功能不中断、只是当次唤醒变慢。
- **存储** 二进制 30 MB + 模型 5.2 MB。

附录 D：语音链路现场快验（1 分钟）

要看什么	命令 / 判据
进程与引擎	日志首行应含 `asr=paraformer`；`ps w | grep [v]oice_assistant.py` 有 1 个进程
最近一轮是否通	日志出现 `WAKE` → `[TURN=…]` `ROUTER` → `RESPONSE` → `TTS_DONE` 四段齐全
本地快路径是否命中	报时/算术应出现 `time_fast_ok`，且无 LLM 调用记录
麦克风增益	`amixer -c 2 sget Mic` 末行应为 `Capture 190 [75%] [on]`
是否处于休眠态	日志出现 `[miss] sleep` 说明在休眠，需完整说「你好小皮」唤回
一键全检	笔记本执行 `bash ~/demo_boot_check.sh`（7 步：网线→看板→摄像头→视觉→语音→校时→温度→音频）

判据都来自 voice.log 的实际输出格式，不含推测；任一条不符，先按附录 B 的 60 秒定位表查证据，再改参数。

源文件：~/RG660MK-EU-AI-CPE/docs/UGREEN智能音箱_设计方案与问题史_20261107.html（可改后重出）｜数据口径：延迟取 voice.log 的 LATENCY 行（样本量已标注），问题史取 git 提交历史与本机验证记录｜本文不含推算值；未实测的项目明确标注「未验证」。